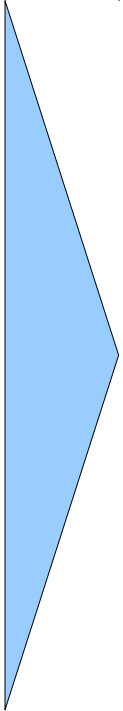
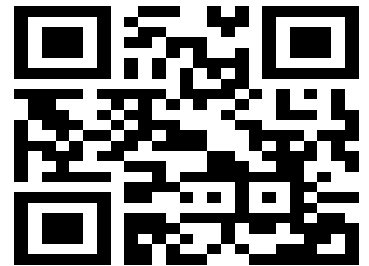
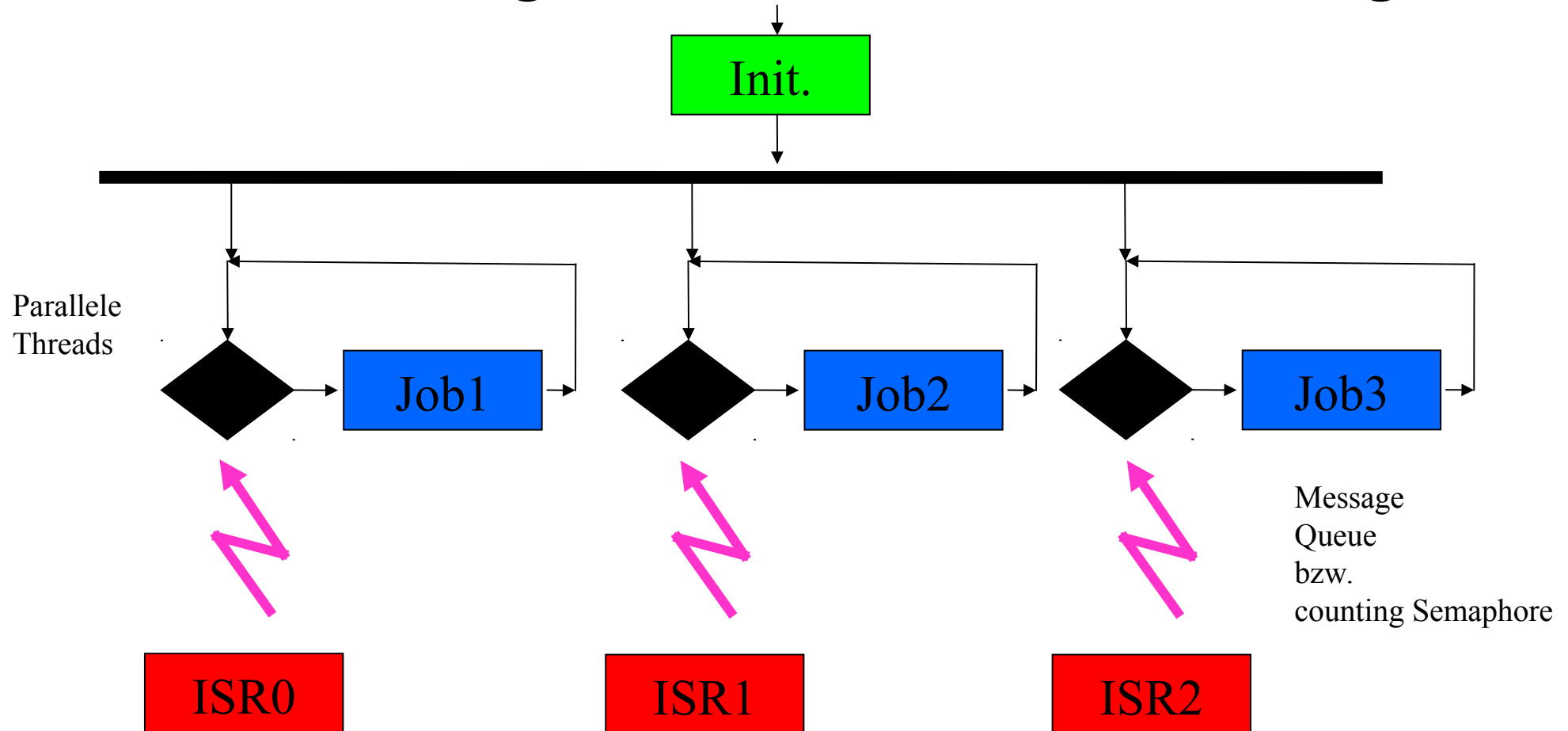


Lecture Overview

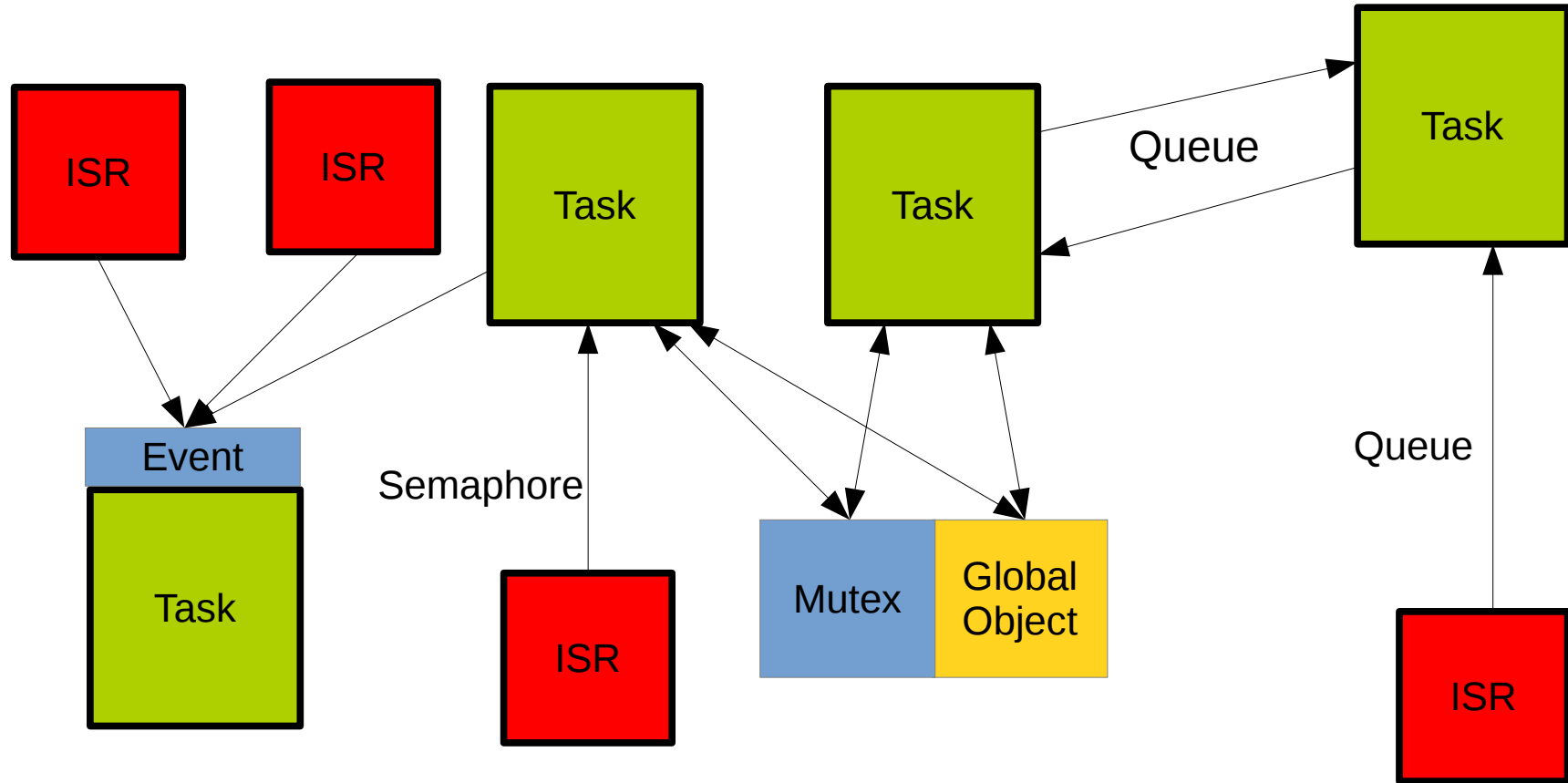
- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats,
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns



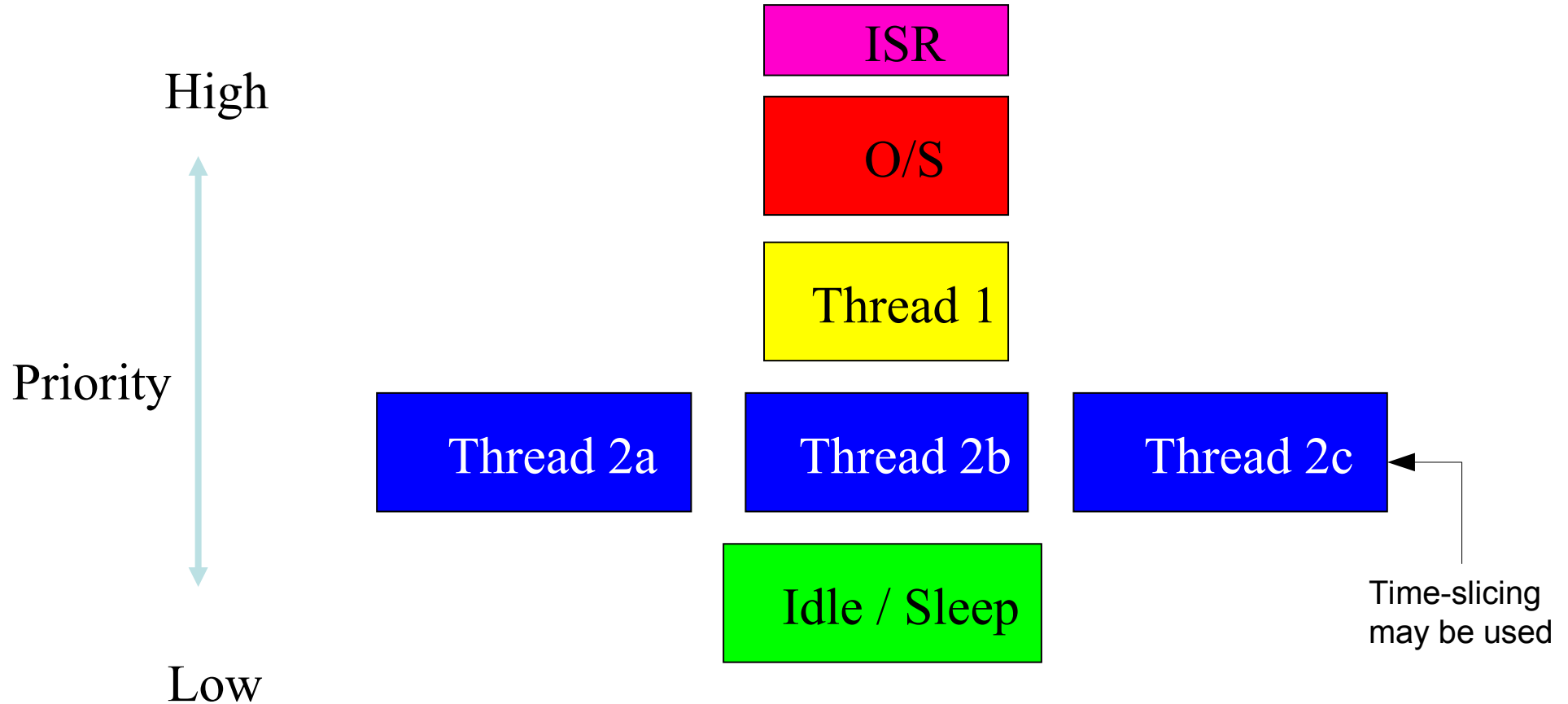
Interrupt + Threadgesteuerte Verarbeitung



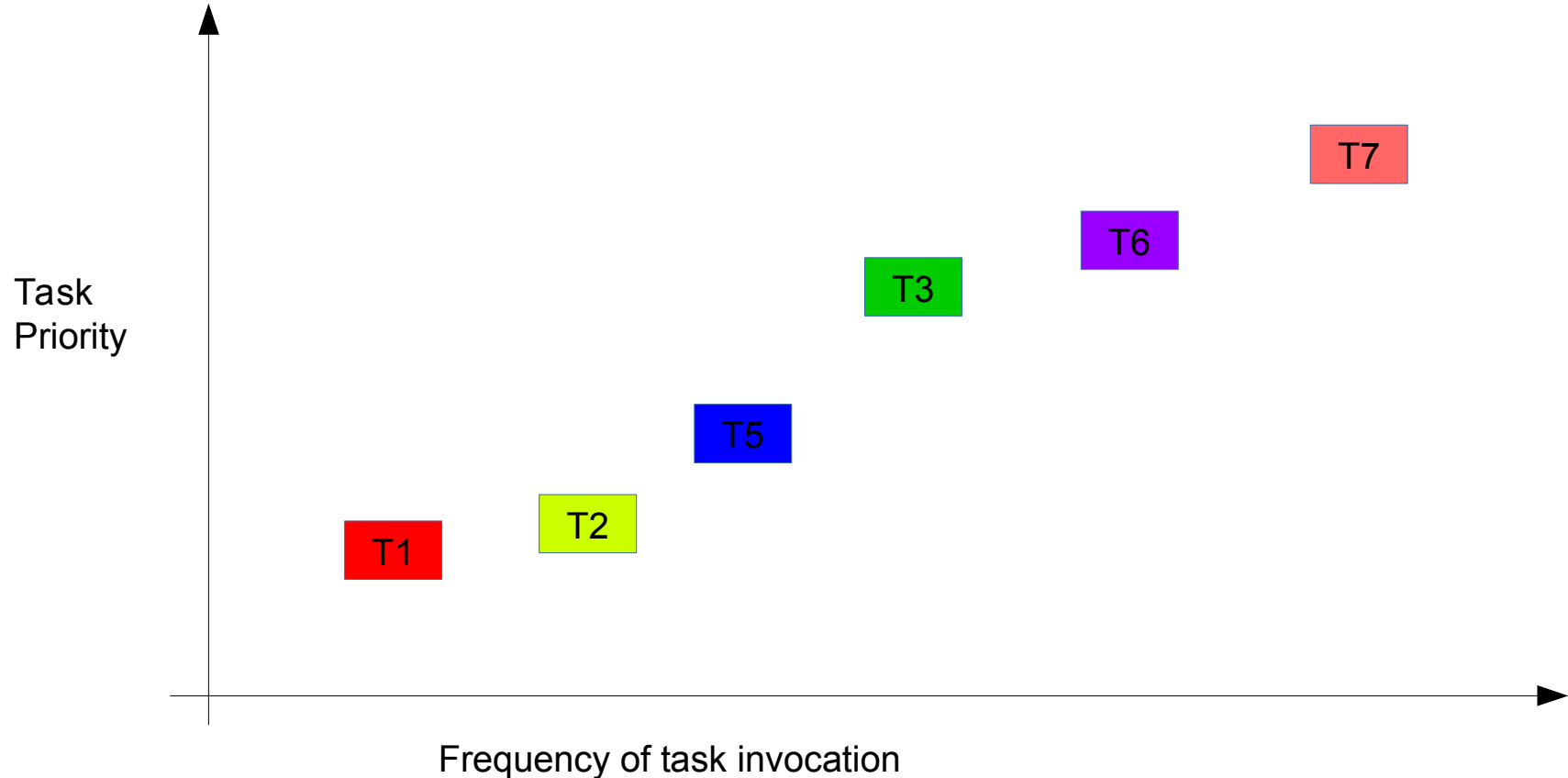
RTOS System: Object Model



Priority-driven Scheduling



Highest Rate First Rate Monotonic Scheduling



Shortest Job First

- Sort all jobs by consumed time until next blockage
- Always select next shortest job for execution
- Problem: Execution time usually “almost” unknown !

EDF = Earliest Deadline First

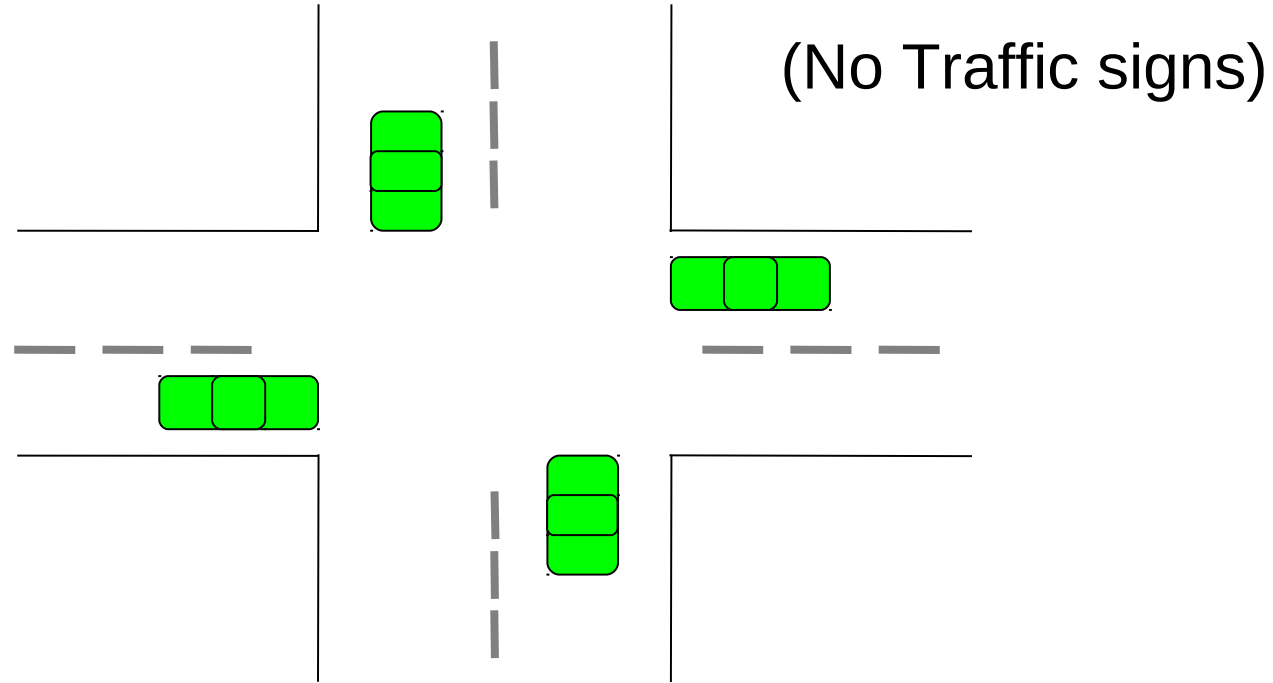
Always activate the task with the shortest time to it's deadline

Problems:

How to find the time until deadline ?

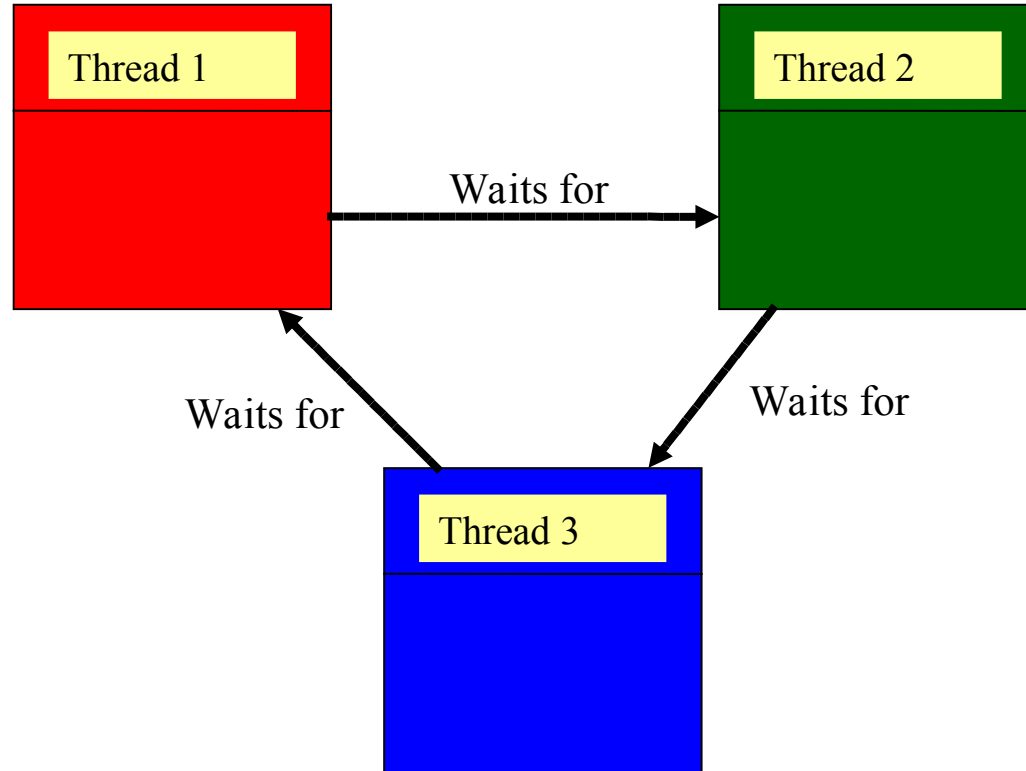
"Starvation" of other jobs still possible

Traffic Deadlock



Cyclic Lock Situation = „Deadlock”

Deadlock



Deadlock

Conditions for Deadlock:

- Threads claim exclusive control of resources
- Threads hold resources while waiting for other resources
- System can not forcibly remove resource locks
- A circular wait condition exists

Escaping Deadlock:

- Preventing conditions with proper programming techniques
- Using lock mechanisms with **timeout**

Priority Inversion

Thread 3
locks resource

Thread 1
preempts thread 3
trying to lock
resource, blocking

Thread 2 running,
preventing thread 3
to release resource

Thread 1



Thread 2



Thread 3



Problem:
Thread 3 blocks
Thread 1 !

Priority-Inheritance

The RTOS knows about tasks waiting on a Mutex
So the RTOS can dynamically execute tasks that have successfully locked the Mutex with the inherited priority of the tasks that have been blocked.

Obviously this creates some overhead to maintain a list of waiting tasks with their priorities.

Priority – Inversion will be avoided !

FreeRTOS: Mutexes use priority-inheritance

Ceiling Priority Protocol

“Priority-Inheritance” has some drawbacks:

- Eventually chains of blocking tasks will be formed
- Deadlocks are possible
- The performance of the kernel may be sub-optimal as there is a quite large overhead with bookkeeping of the priorities

Ceiling Priority Protocol “CPP”

- High priority tasks can only be blocked out by a lower priority task for the time of a single resource access.
- Deadlocks are impossible
- Situations like A blocks B blocks C ... are impossible
- Mutexes are superfluous

“ICPP”

Immediate Ceiling Priority Protocol

For every resource the task with the highest priority that wants to use the resource any time is searched

This task's priority is the **ceiling priority**.

If an active task wants to access a resource and if its dynamic priority is presently higher than the priority of all presently locked resources then the access is granted and the priority is lifted up to the **ceiling priority** for the time of the access.

Original Ceiling Priority Protocol

“OCP”

For every resource the task with the highest priority that wants to use the resource is searched
This task's priority is the **ceiling priority**.

If a task wants to access a resource and if its priority is presently higher than the ceiling priority of all the currently locked resources
then the access will be granted and the scheduler records the ceiling priority of this resource.

If now another task wants to access the resource then the priority of the first task will be lifted up to the ceiling priority for the time of the access.

Lecture Overview

- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats,
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

